

Claude Code in Docker on an Apple MacBook

A reproducible setup guide — standalone container, Docker Compose

Computer Deconstruction Laboratory

17 September 2026

1. What this builds

A single Debian-based Docker image with Claude Code, Node, Python and the usual command-line tooling installed, launched from Terminal with `docker compose`. Your project directory is bind-mounted into the container, so Claude edits the real files on your Mac; everything Claude *runs* happens inside the container.

This is the standalone pattern — the same shape as the Ubuntu setup, not the VS Code `.devcontainer` pattern. No editor integration is required, and the container is equally usable from Terminal, iTerm2, or a VS Code integrated terminal that happens to be pointed at the folder.

macOS (host)	Docker Desktop VM
-----	-----
~/Projects/my-app	---bind---> /home/node/workspace
Safari / Chrome	claude (CLI) node python3 git gh
docker compose	non-root user "node" (uid 1000)
	/home/node/.claude <-- named volume

Two things deserve attention up front, because they are the two that bite:

- **Ownership of `/home/node/.claude`**. Docker seeds a new named volume from whatever exists at the mount point in the image, *including its ownership*. If that directory does not exist in the image, the volume is created root-owned, the `node` user cannot write `.credentials.json`, and every sign-in appears to succeed and then silently fails to persist. The Dockerfile creates and chowns it for exactly this reason.
- **`CLAUDE_CONFIG_DIR`**. Claude Code keeps the token, settings and session history in `~/.claude`, but keeps your OAuth account, personal MCP servers and per-project trust in `~/.claude.json` — a *separate file outside that directory*. Mounting a volume on `~/.claude` alone does not keep you signed in. Setting `CLAUDE_CONFIG_DIR` to the same path moves `.claude.json` inside the volume, and then it does.

2. Prerequisites

Install Docker Desktop for Mac:

```
brew install --cask docker
open -a Docker
```

Then in **Docker Desktop → Settings → Resources**, give the VM enough headroom — 8 GB memory and 4 CPUs is a reasonable floor if Claude will be compiling anything. The `mem_limit` and `cpus` in the compose file cannot exceed what the VM itself has.

In **Settings → General**, confirm the file sharing implementation is **VirtioFS** (the default on current versions). gRPC FUSE is markedly slower on bind mounts.

Verify:

```
docker --version
docker compose version
docker run --rm hello-world
uname -m           # arm64 on Apple Silicon, x86_64 on Intel
```

Alternatives. OrbStack (`brew install --cask orbstack`) is a drop-in replacement with noticeably lower idle overhead and faster bind mounts; Colima (`brew install colima docker docker-compose`) is the free CLI-only option. Every command in this guide works unchanged on all three — they all provide the same `docker` and `docker compose` interface.

3. Project layout

```
mkdir -p ~/claude-docker/workspace
cd ~/claude-docker
```

Place the three files from this bundle here:

```
~/claude-docker/
Dockerfile
docker-compose.yml
cc                # optional launcher script
workspace/        # default project mount point
```

4. The Dockerfile

Key points, top to bottom:

- **No `--platform` anywhere.** On Apple Silicon this builds native `linux/arm64`. Do not force `linux/amd64` unless a specific tool in your toolchain has no arm64 build — that route runs under emulation and is several times slower for compile-heavy work.
- `node:22-bookworm` brings Node and npm and already defines a non-root `node` user at uid 1000. Debian bookworm keeps `apt` familiar if you are coming from the Ubuntu container.
- `npm global prefix` is set to a directory the `node` user owns, so `npm install -g` and Claude Code's own auto-updater work without `sudo`.
- `~/claude` is created and chowned before `USER node` — see section 1.
- `CLAUDE_CONFIG_DIR` is set in the image as well as in compose, so the container behaves correctly even if someone runs `docker run` directly.
- `CLAUDE_CODE_VERSION` is a build arg. Leave it `latest` for everyday use; pin it (e.g. `2.0.14`) plus `DISABLE_AUTOUPDATER=1` when you need a build that is byte-identical months from now — which matters if you are handing this to a class.

Add your own toolchain to the `apt-get install` block: `gcc-arm-none-eabi`, `cmake`, `ninja-build`, and so on. Note that USB device passthrough for microcontroller flashing does **not** work on Docker Desktop for Mac the way it does on a Linux host — the containers run inside a VM with no USB bus. Flash from macOS, and use the container for editing, building and Claude.

5. The compose file

The parts that matter:

Setting	Why
<code>claude-config:/home/node/.claude</code>	Named volume. Survives image rebuild and deletion. This is your login.
<code>./workspace:/home/node/workspace:cached</code>	Your code. Keep the left side as narrow as you can — everything under it is reachable by the agent.
<code>CLAUDE_CONFIG_DIR</code>	Pulls <code>.claude.json</code> into the volume. Without it you re-authenticate constantly.
<code>claude-npm-cache</code>	Keeps npm downloads off the bind mount.
<code>init: true</code>	A real PID 1, so Ctrl-C and cleanup behave.
<code>stdin_open</code> / <code>tty</code>	Claude Code is an interactive TUI; without both you get a dead terminal.
<code>mem_limit</code> / <code>cpus</code>	Guardrails, capped by the Docker Desktop VM's own allocation.

6. Build, run, authenticate

```
cd ~/claude-docker
docker compose build
docker compose run --rm claude
```

You land in a zsh prompt inside the container at `/home/node/workspace`. Confirm the install and, importantly, the ownership:

```
whoami          # -> node
claude --version
ls -ld ~/.claude # -> drwxr-xr-x ... node node (NOT root root)
echo $CLAUDE_CONFIG_DIR # -> /home/node/.claude
```

If that `ls -ld` shows `root root`, stop and fix it now rather than fighting authentication later:

```
exit
docker compose down -v          # deletes the volume
docker compose build --no-cache
docker compose run --rm claude
```

Then start Claude Code:

```
claude
```

Three ways to authenticate

A. Paste the code (simplest, and the one that always works). `claude` prints a sign-in URL. Copy it into Safari or Chrome *on the Mac*, complete the sign-in, and the browser shows a code. Paste that back at the `Paste code here if prompted` prompt. The localhost callback cannot reach into the container, so this manual hand-off is the normal path, not a failure.

B. Pre-generated token (best for unattended or classroom use). If you have Claude Code installed on macOS itself:

```
claude setup-token          # run on the Mac, prints a long-lived token
export CLAUDE_CODE_OAUTH_TOKEN='...'
docker compose run --rm claude claude
```

Uncomment the `CLAUDE_CODE_OAUTH_TOKEN` line in the compose file to pass it through. Treat that token like a password.

C. API key. Set `ANTHROPIC_API_KEY` in the environment if you are billing through the Console rather than a Claude subscription.

What you cannot do

You cannot copy credentials from the macOS host into the container. On macOS Claude Code stores its token in the login Keychain; inside a Linux container there is no Keychain, so it falls back to `~/.claude/.credentials.json`. The two formats and locations are not interchangeable. Authenticate inside the container once, and the volume keeps it.

Verify persistence before you trust it:

```
exit
docker compose run --rm claude claude --version # should not re-prompt
```

7. The `cc` launcher

`docker compose run` from `~/claude-docker` always mounts `~/claude-docker/workspace`, which is inconvenient when your projects live elsewhere. The `cc` script mounts whatever directory you invoke it from:

```
chmod +x ~/claude-docker/cc
sudo ln -sf ~/claude-docker/cc /opt/homebrew/bin/cc # Apple Silicon
# sudo ln -sf ~/claude-docker/cc /usr/local/bin/cc   # Intel
```

Then:

```
cd ~/Projects/my-app
cc claude          # Claude Code, scoped to this project
cc                 # just a shell in the container, same mount
```

8. What persists, and backing it up

Lives in	Survives <code>build --no-cache</code>	Survives <code>down -v</code>
Image layers (installed packages)	no	n/a
<code>claude-config</code> volume (auth, settings, history)	yes	no
Bind-mounted project files	yes	yes (they are on the Mac)

So: `docker compose down` is safe, `docker compose down -v` signs you out.

Back the volume up — this is the piece that is genuinely annoying to reconstruct:

```
docker run --rm \
  -v claude-docker_claude-config:/src:ro \
  -v "$HOME/Backups":/dst \
  alpine tar czf /dst/claude-config-$(date +%Y%m%d).tgz -C /src .
```

Restore:

```
docker run --rm \
  -v claude-docker_claude-config:/dst \
  -v "$HOME/Backups":/src:ro \
  alpine sh -c "cd /dst && tar xzf /src/claude-config-YYYYMMDD.tgz"
```

Confirm the volume's real name first with `docker volume ls` — Compose prefixes it with the project directory name.

Also export the image itself if you want a true offline backup:

```
docker save claude-code-dev:local | gzip > ~/Backups/claude-code-dev.tgz
# restore: gunzip -c ~/Backups/claude-code-dev.tgz | docker load
```

9. macOS-specific notes

Architecture. Native arm64 throughout. If you ever see `exec format error`, something forced an amd64 image; remove the `platform` key or rebuild without `--platform`.

Bind mount speed. VirtioFS is good but not free. For JavaScript projects, uncomment the `claude-node-modules` volume so `node_modules` lives in the VM rather than crossing the file-sharing boundary — build times often halve. C, C++ and Python projects rarely need this.

File watching. `inotify` events do not always propagate across the mount. If a watcher or hot-reloader inside the container misses changes, switch it to polling (`CHOKIDAR_USEPOLLING=1`, `--poll`, or the equivalent).

USB / serial devices. Not available. Docker Desktop for Mac has no USB passthrough, so no flashing a Pico or ESP32 from inside the container. Build in the container, flash from macOS.

localhost. A server started inside the container is not on the Mac's `localhost` unless you publish its port. Add a `ports:` entry to the compose file for anything you want to open in a Mac browser.

No systemd. Same as your Ubuntu container: nothing `systemctl` will work. Start background services explicitly, or add them as separate compose services.

Disk. Docker Desktop's VM disk grows and does not shrink on its own. Run `docker system df` occasionally and `docker system prune` when it bloats — but never `docker system prune --volumes` unless you have the backup from section 8.

10. How this differs from your other two setups

	Windows 11 / WSL2	Ubuntu host	macOS (this)
Engine	Docker Desktop + WSL2 backend	native Docker Engine	Docker Desktop VM
uid/gid mismatch	occasional	the main pain point	not an issue — the file-sharing layer maps ownership
Bind mount speed	poor across <code>/mnt/c</code> , good inside WSL	native	good with VirtioFS
USB passthrough	possible with <code>usbipd</code>	native	not available
<code>sudo</code> in devcontainer	needed <code>NOPASSWD</code> fix	needed <code>NOPASSWD</code> fix	same <code>NOPASSWD</code> fix, included

The Ubuntu directory-ownership problem — the one that produced the persistent authentication failures — does not recur here in the same form, because Docker Desktop’s file-sharing layer presents bind-mounted files as owned by the container user regardless of the host uid. The *volume* seeding problem from section 1 is a different bug with the same symptom, and it does apply on macOS. That is why the Dockerfile creates `~/.claude` explicitly.

11. Optional hardening

Network egress firewall. Anthropic’s reference devcontainer includes `init-firewall.sh`, which restricts outbound traffic to an allowlist. It needs the `NET_ADMIN` and `NET_RAW` capabilities:

```
cap_add:
  - NET_ADMIN
  - NET_RAW
```

It is not required for Claude Code to function, and is only worth the setup if you plan to run with permission prompts disabled.

Skiping permission prompts. Because the container runs as a non-root user and confines execution, `claude --dangerously-skip-permissions` is a defensible choice here — the CLI refuses the flag as root, which is one more reason the image drops to `node`. Understand what it gives up: Claude can still write anything under the bind mount (which is real files on your Mac) and reach anything the network allows. Pair it with the firewall above, or use auto mode instead for fewer prompts without disabling the checks.

Organization policy. Claude Code reads `/etc/claude-code/managed-settings.json` at the highest precedence in the settings hierarchy. For a class where students should not be able to override permission rules, bake it in:

```
RUN mkdir -p /etc/claude-code
COPY managed-settings.json /etc/claude-code/managed-settings.json
```

Anyone with write access to the repo can edit that step out, so it is guidance rather than enforcement.

Do not mount host secrets. Keep `~/.ssh` and cloud credential files out of the container. Use `gh auth login` inside the container, or a repo-scoped token, so a compromise is bounded.

12. Troubleshooting

Symptom	Cause	Fix
Sign-in appears to work, gone next run	<code>~/.claude</code> volume is root-owned	<code>docker compose down -v</code> , rebuild — the Dockerfile's <code>mkdir / chown</code> seeds it correctly
Re-prompted for login on every rebuild	<code>CLAUDE_CONFIG_DIR</code> unset, so <code>.claude.json</code> is outside the volume	Confirm <code>echo \$CLAUDE_CONFIG_DIR</code> inside the container
Browser sign-in never returns	Callback cannot reach into the container	Use the paste-the-code path (6A) or a token (6B)
<code>EACCES</code> on <code>npm install -g</code>	npm prefix not owned by <code>node</code>	<code>NPM_CONFIG_PREFIX</code> block in the Dockerfile
<code>--dangerously-skip-permissions</code> refused	Running as root	Confirm <code>whoami</code> returns <code>node</code>
Garbled TUI, no keyboard input	Missing tty/stdin	Use <code>docker compose run</code> (not <code>up</code>), keep <code>tty</code> and <code>stdin_open</code>
<code>exec format error</code>	amd64 image on arm64	Rebuild without <code>--platform</code>
Builds crawl, fans spin	Emulation, or too little VM memory	Check <code>docker image inspect</code> architecture; raise Docker Desktop resources
Watcher misses file changes	inotify across the mount	Switch the watcher to polling
Cannot reach a container server from Safari	Port not published	Add <code>ports:</code> to the compose file

13. Verification checklist

Run through this once after building; it catches every failure mode above.

```
docker compose run --rm claude sh -c '
  echo "user:    ${whoami}"
  echo "arch:    ${uname -m}"
  echo "config:  $CLAUDE_CONFIG_DIR"
  echo "claude:  ${claude --version}"
  echo "node:    ${node --version}"
  echo "git:      ${git --version}"
  ls -ld $HOME/.claude
  touch $HOME/.claude/.writetest && echo "write:  OK" && rm $HOME/.claude/.writetest
'
```

Expected: user `node`, arch `aarch64` on Apple Silicon, config dir `/home/node/.claude`, that directory owned `node node`, and `write: OK`.

Then confirm persistence by exiting, running again, and checking that Claude Code does not ask you to sign in.